

Caching Strategies Part 1

Prepared By: Dr. D. P. Singh

Caching Overview

- Generally, users expect the **applications to run faster** with quick responses.
- **Caching** is used to implement a **high-speed system** with a large number of users. A cache is a high-speed data storage that stores data temporarily **to serve future requests faster**.
- Database caching is a mechanism that stores frequently accessed data in temporary memory.
- Whenever the **application requests** the data again, that can quickly **get it from the cache**, instead of from the main database.
- Cache helps to **reduce the database workloads**. So, it increases system speed by reducing the need to fetch data from DB.

Importance of Database caching

- **Increase Data Availability:** Caching increases data availability if the primary DB goes down, caching acts as a backup source.
- **Improved System Performance:** In general, **the frequently used data** is stored in the cache. So, the applications can retrieve that data more quickly from the cache instead of the primary DB.
- **Cost Savings:** The caching mechanism reduces DB server load. Caching can help to build the **infrastructure with lower costs compared to DB servers**.
- **Improved User Experience:** Using the Caching technique **responses are returned faster**. It leads to a better overall user experience, particularly for applications that require real-time or near-real-time access to data.

Reading strategies

- Cache-Aside
- Read-Through

Cache-Aside

- Here the **application is responsible for managing** the cache. Whenever the data requests come, the application checks the cache at first.
- If the asked data is available in the cache, simply return it.
- Otherwise, data is retrieved from the database and stored in the cache for future usage. It is also called **Lazy Loading**.

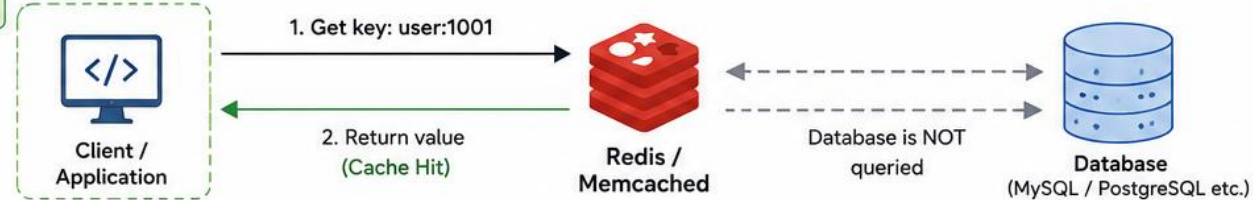
Cache-Aside

Cache-Aside Pattern using Redis (or Memcached)

Application checks **Redis (or Memcached)** first. On cache miss, read from **Database** and store in cache.

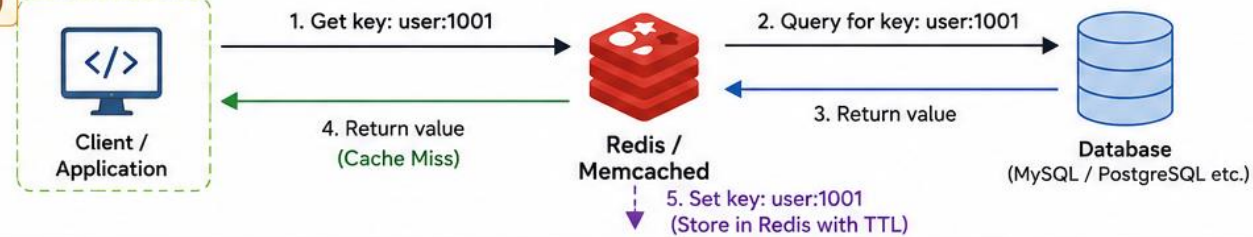
 **Redis / Memcached**
(In-Memory Cache)

1. Read (Cache Hit)



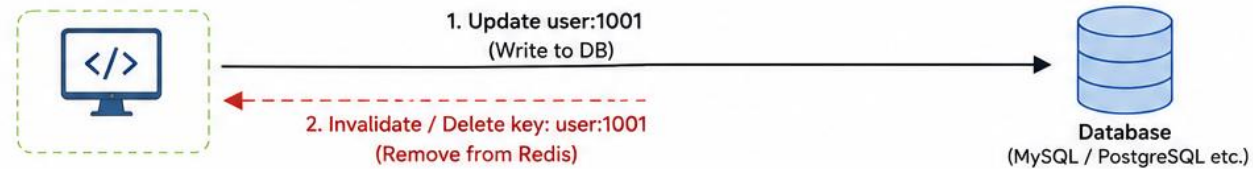
✓ Data returned from Redis.
Fast response.

2. Read (Cache Miss)



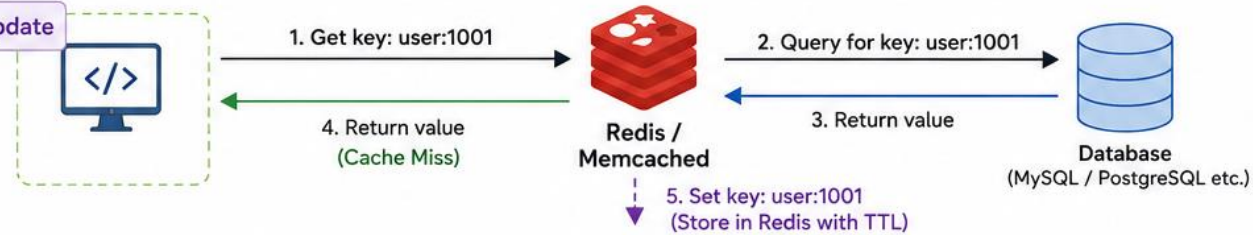
! Data not in cache.
Fetched from DB,
stored in Redis with TTL
and returned to client.

3. Write / Update



✎ Write to database first,
then invalidate cache
so next read gets
fresh data.

4. Next Read After Update



↻ Cache was invalidated,
so it's a miss.
Data fetched from DB
again and cached.



Key Points

- App checks cache first (Redis/Memcached).
- On miss: read from DB, store in cache, return to client.
- On write/update: write to DB, then invalidate cache.
- Simple, scalable and widely used pattern.

Common in Production with:

 **redis** or  **MEMCACHED**

Advantages of Cache-Aside (Lazy Loading)

- **Simple to Implement**
 - **Application logic controls** when data is loaded into cache.
 - Supported easily with Redis or Memcached.
- **Reduces Database Load**
 - Frequently accessed data is served from cache.
 - **Fewer queries reach the database.**
- **Improves Read Performance**
 - Cache access is much faster than database access.
 - **Lower response times for users.**
- **Efficient Memory Usage**
 - Only data that is actually requested gets cached.
 - No need to preload everything.

Advantages of Cache-Aside (Lazy Loading)

Works Well for Read-Heavy Applications

- Ideal for **user profiles, product catalogs, content pages, session data**, etc.

•Cache Failure Doesn't Break the System

- If Redis/Memcached goes down, the application can still **fetch data from the database**.

Disadvantages of Cache-Aside

- **Cache Miss Latency**
 - First request for a key must go to the database.
 - User experiences **slower response on cache miss**.
- **Stale Data Risk**
 - Cache may contain **outdated data if invalidation is not handled correctly**.
 - Common challenge in distributed systems.
- **Additional Application Complexity**
 - Developers must implement:
 - Cache lookup
 - Cache population
 - Cache invalidation
 - TTL management
- **Thundering Herd Problem**
 - If a **popular key expires, many requests may simultaneously hit the database**.
 - Can overwhelm the backend.

Disadvantages of Cache-Aside

Higher Write Complexity

- On updates:
 1. Update database
 2. Invalidate cache
- Incorrect ordering can cause inconsistencies.
- **Cold Start Issues**
 - After cache restart, all requests become cache misses until cache warms up.

When Cache-Aside is a Good Choice

- ✓ Read-heavy workloads
- ✓ User profiles and account data
- ✓ Product catalogs
- ✓ Content management systems
- ✓ Microservices with Redis

When It May Not Be Ideal

- ✗ Extremely **write-heavy systems**
- ✗ Strong consistency requirements (e.g., **banking transactions**)
- ✗ Real-time collaborative applications where **stale data is unacceptable**
- **Rule of thumb:** Cache-Aside is the most widely used caching strategy because it provides a good **balance between performance, simplicity, and scalability**, but **cache invalidation and stale-data handling** are its biggest challenges.

- Thank You